

Docker Container



Created by Shridhar Galande

Docker

Agenda:

- Container
- Traditional vs. Containers vs. Virtual Machines
- Overview of Docker Platform
- Docker Containers Concepts
- Docker Architecture
- Docker Commands
- Dockerfile Commands
- Docker Installation
- Building own docker image
- Docker Compose
- Docker registry - Docker Hub
- Networking inside single docker container

What is a Container?

- A Container is a lightweight, standalone, executable package that includes everything needed to run a piece of software, including the code, runtime, system tools, libraries, and settings.
- It isolates the application from the host system, providing a consistent runtime environment.

Key Concepts:

✓ Isolation:

- Containers run in isolation from one another and from the host system. This means that each container operates as if it's the only process on the machine, avoiding conflicts with other applications or system components.

✓ Portability:

- Containers encapsulate everything an application needs to run, making them highly portable.
- You can develop your application in a container on your local machine and then run it in the same container on any other machine, from a developer's laptop to a production server or a cloud environment.

✓ Lightweight:

- Unlike virtual machines, containers share the host system's OS kernel. This makes them much more lightweight, using fewer resources and allowing many containers to run on a single host without the overhead of full operating system instances.

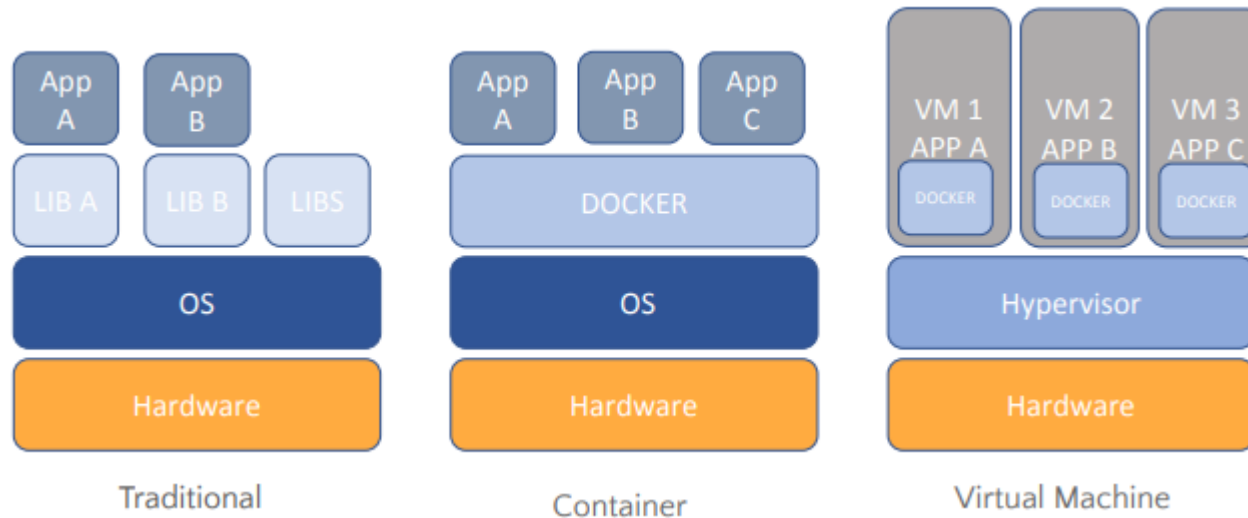
✓ Consistency:

- Containers ensure that an application behaves the same regardless of where it's deployed. This eliminates issues that arise due to differences in operating systems, dependencies, or environment configurations.

✓ Efficiency:

- Because containers share the OS kernel, they start up almost instantly and use far less memory and storage compared to traditional virtual machines.

Traditional vs. Containers vs. Virtual Machines



Feature	Traditional Deployment	Containers	Virtual Machines (VMs)
Isolation	None	Process-level	Strong (full OS)
Resource Usage	High	Process-level	Strong (full OS)
Portability	Low	High	Medium
Deployment Speed	Slow	Fast (Instant Start)	Moderate (OS boot time)
Scalability	Limited	High	Moderate
Security	Low (shared environment)	High (isolated processes)	High (isolated OS)
Overhead	High (manual setup/config)	Low (shared OS kernel)	High (full OS in each VM)

Overview of Docker Platform

❖ What is Docker?

- **Open Platform:** Docker is an open platform that allows you to develop, ship, and run applications efficiently.
- **Application & Infrastructure Separation:** Docker enables you to separate your applications from your infrastructure, allowing for faster software delivery.

❖ Why Docker?

- ❖ **Infrastructure as Code:** Manage your infrastructure with the same practices you use for your applications, ensuring consistency and speed.
- ❖ **Reduced Time to Production:** By leveraging Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the time from code development to production deployment.

❖ The Docker Platform:

- **Containers as Isolated Environments:** Docker packages applications into containers, providing a loosely isolated environment that enhances security and allows multiple containers to run simultaneously on a single host.
- **Lightweight & Portable:** Containers are lightweight, containing everything needed to run the application, independent of the host's configuration.
- **Consistent & Shareable:** Share containers with confidence, knowing that they will run the same way on any environment.

❖ Lifecycle Management:

- ❖ **Development:** Build and develop your applications and their dependencies within containers.
- ❖ **Distribution & Testing:** Use containers as the unit of distribution and testing, ensuring consistency across environments.
- ❖ **Deployment:** Deploy containers directly into production environments, whether in a local data center, cloud, or hybrid setup, with the same ease and reliability.

Docker Containers Concepts

❖ Docker Image:

- A Docker Image is a lightweight, standalone, and executable software package that includes everything needed to run a piece of software: code, runtime, system tools, libraries, and settings.
- Example: Think of it as a snapshot or a template for creating Docker containers.

❖ Docker Container:

- Docker Container is a running instance of a Docker Image. It's an isolated environment where the application runs, using the code and dependencies defined in the image.
- Example: Similar to a virtual machine but much more lightweight and efficient.

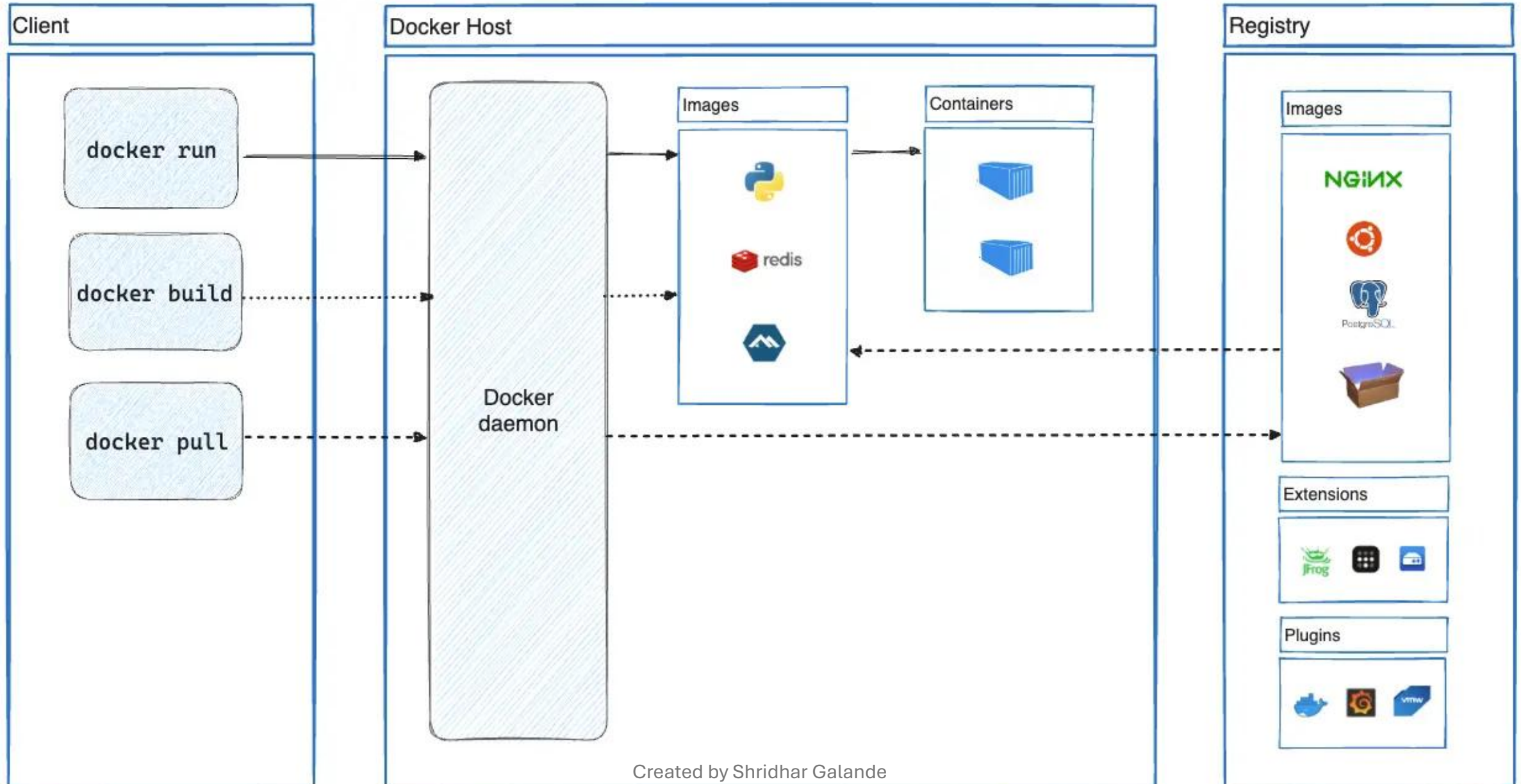
❖ Dockerfile:

- A Dockerfile is a script containing a series of instructions on how to build a Docker Image. It defines the base image, copies files, installs dependencies, and sets up the environment.
- Example: Instructions might include commands like FROM, COPY, RUN, and CMD.

❖ Docker Hub:

- Docker Hub is a cloud-based registry where Docker Images are stored, shared, and distributed. It's a central repository for finding official and community-contributed images.
- Example: You can pull an image from Docker Hub using **`docker pull [image_name]`**

Docker Architecture



Docker Client-Server Architecture

❖ Docker Architecture:

- **Client-Server Model:** Docker uses a client-server architecture where the Docker client communicates with the Docker daemon (dockerd), which handles building, running, and distributing containers.
- **Communication:** The Docker client and daemon communicate via REST API, using UNIX sockets or network interfaces.

❖ Docker Daemon ('dockerd'):

- **Role:** The daemon listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes.
- **Orchestration:** Daemons can communicate with each other to manage multi-container services.

❖ Docker Client ('docker'):

- **Primary Interface:** The Docker client is the main interface for users, where commands like docker run and docker build are sent to the daemon for execution.
- **Multiple Daemons:** The client can communicate with multiple Docker daemons, both local and remote.

❖ Docker Desktop:

- **All-in-One Package:** Docker Desktop is an easy-to-install application that includes Docker Daemon, Docker Client, Docker Compose, Kubernetes, and other tools for containerized development on Mac, Windows, or Linux.

❖ Docker Registries:

- **Image Storage:** Docker registries store Docker images. By default, Docker pulls images from Docker Hub, a public registry. You can also set up your own private registry for storing custom images.

❖ Docker Objects:

- **Key Objects:** Docker manages various objects such as images, containers, networks, volumes, and plugins, which are essential for building and managing containerized applications.

Docker Commands

- ✓ **Check Docker Version:**
 - > docker --version
- ✓ **Pull an Image from Docker Hub:**
 - > docker pull [image_name]
- ✓ **Run a Docker Container:**
 - > docker run [image_name]
- ✓ **List Running Containers:**
 - > docker ps
- ✓ **List All Containers (Running & Stopped):**
 - > docker ps -a
- ✓ **Stop a Running Container:**
 - > docker stop [container_id]
- ✓ **Remove a Container:**
 - > docker rm [container_id]
- ✓ **Remove an Image:**
 - > docker rmi [image_name]
- ✓ **Build an Image from a Dockerfile:**
 - > docker build -t [image_name] .
- ✓ **View Logs of a Container:**
 - > docker logs [container_id]
- ✓ **List Docker Images:**
 - > docker images
- ✓ **Inspect a Container:**
 - > docker inspect [container_id]
- ✓ **Start a Stopped Container:**
 - > docker start [container_id]
- ✓ **Attach to a Running Container:**
 - > docker attach [container_id]
- ✓ **Execute a Command in a Running Container:**
 - > docker exec -it [container_id] [command]

Essential Dockerfile Commands

- **FROM:** Sets the base image for the Docker image.
- **COPY:** Copies files from the host to the container.
- **RUN:** Executes a command during the image build process.
- **CMD:** Specifies the default command to run in the container.
- **EXPOSE:** Indicates the port on which the container listens.
- **ENV:** Sets an environment variable in the container.
- **WORKDIR:** Sets the working directory for subsequent commands.
- **ENTRYPOINT:** Defines the main command to run in the container.
- **VOLUME:** Creates a mount point for data storage, allowing data to persist beyond the container's lifecycle.
- **LABEL:** Adds metadata to the image, such as version information or maintainer details.
- **ADD:** Copies files from the host or a remote URL to the container, with additional functionality for handling archives.
- **ARG:** Defines a build-time variable that can be passed to the Dockerfile at build time.

Docker Installation

[Install Docker Engine on Ubuntu](#)

Building own docker image

Demo

Thank you for your attention.

Happy learning!!!